

Guía de Ejercicios 2

Tecnología Digital VI

Juan Ignacio Elosegui

Universidad Torcuato Di Tella
Abril 2026

Modelos Lineales

Ejercicio 1

- Y_i es la variable dependiente.
- β_0 es el intercepto.
- β_1 es la pendiente del atributo.
- X_i es la observación i del *feature* X .
- ϵ_i es el error aleatorio de la observación i .

Ejercicio 2

No es lo óptimo usar *gradient descent*, porque es un proceso iterativo. Si querés, podés usarlo igual. Lo mejor para resolver un problema de regresión lineal es usar mínimos cuadrados, porque tiene una fórmula cerrada. Seguramente haya excepciones, pero en general conviene la segunda opción.

Ejercicio 3

Sigue siendo un modelo lineal porque se usa la combinación lineal de los atributos almacenada en la variable z en la función sigmoide. La sigmoide solamente es una transformación que se aplica sobre esa combinación lineal.

Ejercicio 4

A ver qué pasa si la combinación lineal es cero, es decir:

$$\begin{aligned} z &= 0 \\ \Rightarrow \sigma(0) &= \frac{1}{1 + e^{-0}} \\ \Rightarrow \sigma(0) &= \frac{1}{1 + 1} \\ \Rightarrow \sigma(0) &= \frac{1}{2} \\ \therefore \sigma(0) &= 0,5 \end{aligned}$$

La probabilidad asignada da 0,5. Esto quiere decir que la clasificación binaria resultante depende de dónde esté ubicada la frontera de decisión.

Ejercicio 5

Para optimizar una regresión logística se suele usar *Log-Loss* porque es continua y derivable, lo que lo hace genial para usar el método de *gradient descent*.

La métrica de evaluación es *Accuracy* -claramente está también *Precision*, *Recall*, etcétera- las cuales no son derivables en todo su dominio, por lo que no sirve para *gradient descent*.

Aunque son distintas, ambas persiguen el mismo objetivo: cuando la *Log-Loss* baja, la *Accuracy* tiende a subir. Minimizar la incertidumbre probabilística del modelo termina mejorando las clasificaciones.

Supongamos dos modelos: A y B.

- **Modelo A.** Predijo las probabilidades $\{0,9, 0,8, 0,9, 0,95\}$, que son todas clasificadas como clase 1. Este modelo tiene un *Accuracy* de 100 %.
- **Modelo B.** Predijo las probabilidades $\{0,5, 0,55, 0,6, 0,5\}$, que son todas clasificadas como clase 1. Este modelo tiene un *Accuracy* de 100 % también.

Tienen la misma *Accuracy* los dos modelos, pero lo que hace la *Log-Loss* es -básicamente- calcular qué tan seguro está el modelo de la clasificación que le puso a una observación. Es como una métrica de calidad de la clasificación. El *Log-Loss* del Modelo A es mucho menor porque sus probabilidades están más cerca de su clase real (1) que el Modelo B.

Ejercicio 6

Una Regresión Logística estándar no puede separar este dataset de manera efectiva, ya que su frontera de decisión es un hiperplano, y no existe ninguna recta capaz de separar un círculo interior de un anillo exterior. Pero, mediante ingeniería de atributos, es posible lograrlo.

Si se introduce un atributo $X_3 = X_1^2 + X_2^2$, este representa la distancia al origen al cuadrado. De esta forma, en el espacio expandido de atributos (X_1, X_2, X_3) , las clases se vuelven linealmente separables y la regresión logística puede encontrar un umbral sobre X_3 que los separe correctamente.

El modelo sigue siendo lineal en sus parámetros $(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3)$, pero al operar sobre atributos con transformaciones no lineales, logra capturar una frontera que en el espacio original es circular.

Árboles de Decisión

Ejercicio 7

En un **modelo lineal**, la frontera de decisión es un único hiperplano que divide todo el espacio de atributos en dos regiones. Esta frontera puede tener cualquier orientación, es decir, puede ser diagonal a los ejes.

En un **árbol de decisión**, este divide el espacio de atributos en rectángulos tras hacer cortes de manera sucesiva de a un atributo por vez. Las fronteras son paralelas a los ejes.

Ejercicio 8

- **Para datos faltantes.** El árbol puede usar **surrogate splits**, que funcionan como *divisiones sustitutas*. Cuando el árbol encuentra que para una observación falta el valor del atributo por la que quiere partir, busca otro atributo que produzca una partición *similar* y usa esa en su lugar.
- **Para atributos categóricos.** En lugar de necesitar *One-Hot Encoding*, el árbol puede partir directamente evaluando *subconjuntos* de categorías. Por ejemplo, si tenemos un atributo **Color** con valores {Rojo, Azul, Verde}, el árbol puede *evaluar particiones* como {Rojo, Azul} vs {Verde}, probando distintas agrupaciones de categorías y *eligiendo la que mejor separe las clases*.

Ejercicio 9

Un índice de *Gini* igual a cero implica que la pureza de ese nodo es *total*, es decir, todas las observaciones dentro del nodo son de la misma clase.

El índice Gini se calcula como:

$$Gini = 1 - \sum_{i=1}^C p_i^2$$

donde p_i es la proporción de muestras de la clase i en el nodo y C es la cantidad de clases. Si todas las muestras pertenecen a una misma clase, entonces una proporción vale 1 y las demás valen 0, por lo que

$$Gini = 1 - 1^2 = 0$$

En el extremo opuesto, si se tienen dos clases perfectamente mezcladas en proporciones 50/50,

$$Gini = 1 - (0,5^2 + 0,5^2) = 0,5$$

que es el valor máximo de impureza para clasificación binaria.

Intuitivamente, el índice Gini mide la **probabilidad de clasificar incorrectamente una muestra si se la etiquetara al azar según la distribución de clases del nodo**. Si todas las muestras son de la misma clase, esa probabilidad es cero.

Ejercicio 10

Consideremos un nodo padre con 200 muestras: 100 de clase A y 100 de clase B (proporción 50/50).

Se evalúa el siguiente split:

- Hijo izquierdo: 60 de clase A, 40 de clase B (100 muestras)
- Hijo derecho: 40 de clase A, 60 de clase B (100 muestras)

El **Missclassification Error** se calcula como $1 - \max(p_i)$, es decir, la proporción de la clase que no es mayoritaria.

$$ME_{\text{padre}} = 1 - \max\left(\frac{100}{200}, \frac{100}{200}\right) = 1 - 0,5 = 0,5$$

$$ME_{\text{izq}} = 1 - \max\left(\frac{60}{100}, \frac{40}{100}\right) = 1 - 0,6 = 0,4$$

$$ME_{\text{der}} = 1 - \max\left(\frac{40}{100}, \frac{60}{100}\right) = 1 - 0,6 = 0,4$$

$$ME_{\text{hijos}} = \frac{100}{200} \times 0,4 + \frac{100}{200} \times 0,4 = 0,4$$

Misclassification Error baja de 0,5 a 0,4. Ahora bien, supongamos que el *split* anterior no está disponible y el mejor *split* posible genera:

- Hijo izquierdo: 50 de clase A, 50 de clase B (100 muestras)
- Hijo derecho: 50 de clase A, 50 de clase B (100 muestras)

$$ME_{\text{izq}} = 1 - 0,5 = 0,5; \quad ME_{\text{der}} = 1 - 0,5 = 0,5$$

$$ME_{\text{hijos}} = 0,5$$

En este caso, *Misclassification Error* no detecta ningún progreso (se mantiene en 0,5), y el árbol no puede avanzar.

Ahora, con el **índice Gini**, volvemos al primer *split* (60A/40B vs 40A/60B):

$$Gini_{\text{padre}} = 1 - (0,5^2 + 0,5^2) = 1 - 0,5 = 0,5$$

$$Gini_{\text{izq}} = 1 - (0,6^2 + 0,4^2) = 1 - 0,52 = 0,48$$

$$Gini_{\text{der}} = 1 - (0,4^2 + 0,6^2) = 1 - 0,52 = 0,48$$

$$Gini_{\text{hijos}} = \frac{100}{200} \times 0,48 + \frac{100}{200} \times 0,48 = 0,48$$

Gini detecta una mejora (de 0,5 a 0,48), lo que permite al árbol seguir creciendo hacia nodos más puros. Esto se debe a que Gini es sensible a los cambios en las proporciones de las clases, mientras que *Misclassification Error* solo se fija en cuál es la clase mayoritaria, lo que lo hace inadecuado como métrica de entrenamiento.

Ejercicio 11

Si se agregan o se elimina un pequeño porcentaje de las muestras, la estructura del árbol puede cambiar completamente, porque el mismo se construye de forma *greedy*: cada nodo elige el mejor *split* posible en ese momento. Por eso se dice que el modelo tiene una alta variabilidad y sensibilidad ante un cambio sutil en los datos.

Ejercicio 12

Si llega $X = 75$, esta observación caerá lo más a la derecha posible del árbol. Incluso, tendrá el mismo tratamiento que aquellas observaciones muy cercanas a $X = 50$. Esto pasa porque el árbol fue entrenado con valores de X entre 0 y 50, por lo que todas las reglas de partición van a tener *thresholds* dentro de ese rango. Es una limitación importante de los árboles de decisión.

Ejercicio 13

Explicarle un modelo de regresión logística puede parecer abstracto para alguien que no esté metido en el tema. Por otro lado, un árbol de decisión de baja profundidad modela perfectamente la toma de decisiones que tenemos los humanos, por lo que es más transparente y más fácil de explicar.

Ensamblajes

Ejercicio 14

Puede haber dos casos en el cual Majority Voting arroje un resultado: dos modelos aciertan o tres modelos aciertan. Esto se puede representar como:

$$P(\text{dos aciertan}) = 0,8 \cdot 0,8 \cdot (1 - 0,8) = 0,512$$

$$P(\text{tres aciertan}) = 0,8 \cdot 0,8 \cdot 0,8 = 0,512$$

Son dos casos disjuntos en los que el ensamble puede acertar, por lo que:

$$P(\text{ensamble acierta}) = 0,512 + 0,512 = 0,896$$

O sea, *Accuracy* subió de 0,8 a 0,896.

Ejercicio 15

Rank Correlation se usa en lugar de Pearson porque los árboles de decisión trabajan con *rankings* y *thresholds*, no con relaciones lineales.

Pearson mide la relación lineal entre dos atributos. Si dos atributos tienen una relación monótona pero no lineal, Pearson puede dar un valor bajo aunque en la práctica contengan información muy similar para un árbol.

En cambio, **Rank Correlation** mide la relación monótona: solo le importa si cuando un atributo sube, el otro también sube (o baja), sin importar si la relación es lineal o no. Y esto es exactamente lo que le importa a un árbol de decisión, porque al hacer *splits* del tipo $X \leq \text{threshold}$, lo único que importa es el orden de los valores, no su magnitud. Si dos atributos tienen el mismo *ranking* (el mismo orden de las observaciones), van a producir exactamente los mismos *splits* y, por lo tanto, son redundantes para el modelo.

Ejercicio 16

- **Paralelizables.** Son aquellos en los cuales cada árbol se entrena de forma independiente sobre su propio subconjunto de datos. Ejemplos: **Bagging** y **Random Forest**.
- **No paralelizables.** Son aquellos en los cuales cada árbol nuevo se entrena para corregir los errores del anterior. Se necesita el resultado del árbol i para poder entrenar el árbol $i + 1$. Ejemplos: **AdaBoost**, **XGBoost**, y cualquier otro método de *boosting*.

Ejercicio 17

Hay dos métodos de interpretabilidad para medir la importancia de los atributos:

1. **Feature Importance.** Se calcula promediando las reducciones de *Gini* que produce cada atributo en todos los *splits* de todos los árboles del bosque. Los atributos que generan mayores reducciones son las más importantes.
2. **Permutation Importance.** En lugar de mirar las reducciones de *Gini*, mide cuánto empeora la *performance* del modelo al desordenar aleatoriamente los valores de cada atributo en el conjunto de test.

Ejercicio 18

En **Random Forest**, cada árbol se entrena de forma independiente sobre un subconjunto aleatorio de datos y atributos. Agregar más árboles es como agregar más “opiniones independientes”: el promedio se estabiliza y la varianza baja, pero nunca se va a llegar al *overfitting* por agregar demasiados. En el peor caso, el rendimiento se estanca pero no empeora.

En **Boosting** (XGBoost, etc.), cada árbol nuevo se entrena específicamente para corregir los errores de los anteriores. Si se agregan demasiados árboles, el modelo empieza a corregir errores que en realidad son ruido normal del *training set*, y termina aprendiendo ese ruido en lugar de aprender patrones generales. Cada árbol adicional le da más capacidad al modelo de ajustarse a los datos, por lo que la cantidad de árboles controla directamente la complejidad del modelo y es un parámetro de *overfitting*.

Ejercicio 19

Existen porque cada una aporta optimizaciones diferentes para mejorar velocidad, manejo de datos o rendimiento:

- **XGBoost** fue la primera implementación optimizada. Introduce regularización en la función de costo (L1 y L2) para controlar el *overfitting*, manejo de datos faltantes integrado, y paralelización en la construcción de cada árbol individual.
- **LightGBM** prioriza la velocidad y eficiencia en datasets grandes. Crece los árboles “*leaf-wise*” (expande la hoja que más reduce el error), lo cual converge más rápido.
- **CatBoost** se especializa en el manejo nativo de atributos categóricos, sin necesidad de hacer OHE ni *label encoding* previo. Además usa una técnica que reduce el sesgo en la estimación de los gradientes y lo hace más robusto al *overfitting*.

Ejercicio 20

Se podrían utilizar ensambles de árboles en datos no tabulares como imágenes, audio o texto, pero en la práctica no se hace porque estos datos requieren que el modelo aprenda representaciones a partir de los datos crudos.

Una imagen es una matriz de píxeles, y las relaciones espaciales entre esos píxeles son las que contienen la información relevante. Los árboles tratan cada atributo de forma independiente, así que no pueden capturar esas estructuras espaciales, temporales o secuenciales.

Las redes neuronales sí están diseñadas para capturar estas estructuras: aprenden filtros, patrones locales y dependencias secuenciales de forma jerárquica. Por eso dominan en datos no tabulares.

Casos integrales

Ejercicio 21

Qué es lo que haría:

1. **Problema de clasificación.** Si un cliente cancela es $Churn = 1$, caso contrario es $Churn = 0$. Hay que definir bien a qué se considera *churn*, si cancela antes del vencimiento o no renueva su suscripción, por ejemplo.

2. **Preprocesamiento de atributos.** El dataset tiene atributos numéricos y categóricos, así que el preprocesamiento depende del modelo elegido.

Para modelos lineales, se necesita escalar los atributos numéricos y codificar los atributos categóricos con OHE.

Para modelos basados en árboles como XGBoost o LightGBM, se pueden usar con menos preprocesamiento ya que manejan atributos categóricos nativamente.

Los datos faltantes se pueden imputar con la media, mediana o moda.

Opcionalmente, se puede hacer ingeniería de atributos si se quisiera.

3. **Desbalance.** Como hay muchos más clientes que no cancelan, el modelo podría tender a predecir siempre $Churn = 0$ y tener un *Accuracy* alto pero inútil. Para manejar esto se puede usar *oversampling* de la clase minoritaria con SMOTE, *undersampling* de la mayoritaria o usar métricas que no sean *Accuracy*.

4. **Modelos a utilizar.** Se podría arrancar con un modelo simple como Regresión Logística y luego probar modelos más potentes como Random Forest o XGBoost.

Lo ideal es comparar varios y elegir el mejor mediante *cross-validation*.

5. **Métricas de evaluación.** Dado el desbalance, *Accuracy* no es una buena métrica. Conviene usar *Precision*, *Recall*, F1-Score y AUC-ROC.

La elección entre priorizar *Precision* o *Recall* depende del negocio: si el costo de perder un cliente es alto, conviene priorizar *Recall* (detectar la mayor cantidad de *churn* posible, aunque haya algunos falsos positivos). Si las acciones de retención son costosas, conviene priorizar *Precision*.

6. **Consideraciones.** Hay que definir cada cuánto se reentrena el modelo, ya que los patrones de *churn* pueden cambiar con el tiempo.

También hay que monitorear las métricas del modelo en producción y definir un *threshold* de probabilidad adecuado para clasificar.

Además, es importante la interpretabilidad para entender qué factores generan *churn* y poder diseñar estrategias de retención específicas. (Este punto fue promptado fuertemente)

Ejercicio 22

Los problemas que identifiqué son:

1. **Desbalance y métrica inadecuada.** De 100.000 préstamos, solo 3.500 son *default*. Un modelo que prediga siempre “no default” tendría un 96,5 % de *Accuracy*, que es lo que reportan. Seguro que el modelo no está aprendiendo nada y solamente prediga la clase mayoritaria.

La solución es usar otras métricas adecuadas (como en el ejercicio anterior), y aplicar técnicas para manejar el desbalance.

2. **Modelo muy complejo.** Una Red Neuronal Profunda con 5 capas ocultas es demasiado como para un problema tabular con probablemente pocos atributos.
Con 100.000 observaciones y datos tabulares, otros modelos (como XGBoost, LightGBM o incluso Regresión Logística) pueden funcionar mejor y entrenarse más rápido.
3. **Interpretabilidad.** El área legal recibe denuncias de usuarios que quieren saber por qué fueron rechazados, y el equipo no puede explicarlo.
4. **Atributos extraños.** Cuesta entender por qué consideraron al atributo “modelo del celular” como uno relevante. Se deberían usar atributos más directamente relacionadas con la capacidad de pago.
5. **Métrica de optimización.** Usaron *Binary Cross-Entropy* como función de pérdida, pero no ajustaron el *threshold* de decisión ni manejaron el desbalance, así que el modelo optimizó la función sin aprender a detectar *defaults*.
6. **Consideraciones.** ¿Habrán hecho alguna validación del modelo? ¿Cuál es el *threshold* para predecir si un cliente va a entrar en *default*?

Ejercicio 23

Respondiendo a las preguntas guía:

- **Categoría de ML.** Es aprendizaje supervisado, ya que se tienen reportes históricos de problemas y roturas que actúan como etiquetas.
- **Preprocesamiento de atributos y otros datasets.** Los datos de sensores (vibraciones, sonidos, temperatura, energía) son datos temporales, así que hay que transformarlos en atributos tabulares.

Los datos de fabricación son atributos estáticos que se pueden incorporar directamente. También se pueden crear atributos como horas de uso acumuladas, días desde el último mantenimiento, etc.

Se podrían sumar datos de las órdenes de trabajo de mantenimiento, condiciones ambientales (temperatura, humedad del ambiente de la planta), turnos de operación, datos del operario, etc.

- **Balance.** Es muy probable que las fallas sean eventos raros comparados con la operación normal, generando un desbalance importante. No es necesario explicar otra vez cómo solucionar esos desbalances.

- **Modelos a utilizar.** Estaría bueno empezar con un modelo simple como Regresión Logística, y luego probar modelos de ensamble como Random Forest, XGBoost o LightGBM.

- **Métricas de interés.** Un falso negativo (no detectar una falla) puede causar una rotura mayor con costos enormes: parada de producción, daños a la máquina, riesgo de seguridad.

Un falso positivo (mantenimiento innecesario) tiene un costo menor.

Por eso conviene priorizar *Recall* (detectar la mayor cantidad de fallas posible), aceptando algunos falsos positivos. Las métricas clave serían *Recall*, *Precision*, F1-Score y AUC-ROC.

- **Interpretabilidad.** Saber qué sensor o atributo dispara la alerta ayuda al equipo de mantenimiento a saber qué revisar en la máquina. *Feature Importance* es una herramienta útil para esto.
- **Consideraciones.** El modelo necesita recibir datos de sensores en tiempo real para generar alertas oportunas. Hay que definir cada cuánto se ejecuta la predicción. Se debe monitorear la *performance* del modelo continuamente, ya que las máquinas se degradan con el tiempo y los patrones pueden cambiar.